EXPERIMENT NO: 7

Student Name: Likhil N Maiya **UID:** 23BCS11938

Branch: BE-CSE **Section/Group:** KRG-1B

Semester: 6th **Date of Performance:** 23/03/2026

Subject Name: System Design **Subject Code:** 23CSH-314

Aim

Ola Uber like cab booking application

Objectives

- Design a scalable real-time messaging system similar to Ola/Uber
- Enable instant communication between drivers in your area
- Support text, voice communication
- Ensure high availability and low latency messaging
- Maintain strong privacy and end-to-end encryption
- Handle billions of messages daily efficiently
- Provide seamless user experience across devices
- Support horizontal scaling as user base grows
- Ensure reliable message delivery and synchronization

Procedure

Requirement Analysis

- Identify functional and non-functional needs
- Define scale assumptions (users, messages/sec)

System Architecture Design

- Choose distributed architecture
- Define core services:
  - Authentication Service

- Messaging Service
- Media Service
- Notification Service
- Presence Service

Communication Protocol Design

- Use WebSockets for real-time messaging
- HTTP/HTTPS for APIs Data Storage

Design

- User database
- Message database
- Media object storage
- Cache layer (Redis) Scalability Planning

- Load balancers
- Sharding strategy
- Horizontal scaling Reliability Design

- Message queues
- Retry mechanisms
- Replication Security Design

- Encryption protocols
- Authentication tokens
- Key exchange system

Deployment Design

- Cloud infrastructure
- CDN for media delivery
- Monitoring tools

Functional Requirements

Clients /User Management

1. User registration using phone number
2. OTP verification and authentication
3. Profile creation and updates
4. Contact synchronization
5. User presence status (online/offline/last seen) **Messaging Features**

Communication Features

15. Voice calls
16. Push notifications for new messages

Multi-device Support

27. Sync messages across multiple devices

Non-Functional Requirements

Performance

- Message delivery latency < 100–300 ms
- Support millions of concurrent users
- Fast media upload/download

Scalability

- Horizontal scaling of servers
- Distributed architecture
- Load balancing

Availability

- 99.99% uptime target
- Fault-tolerant services
- Automatic failover mechanisms

Reliability

- Guaranteed message delivery
- No message loss
- Retry mechanisms

Security

- End-to-end encryption
- Secure key management
- Data privacy compliance

Consistency

- Message ordering per conversation

- Eventual consistency across devices

Maintainability

- Modular microservices architecture
- Easy deployment and updates

Observability

- Logging
- Monitoring
- Alerting systems Storage Efficiency

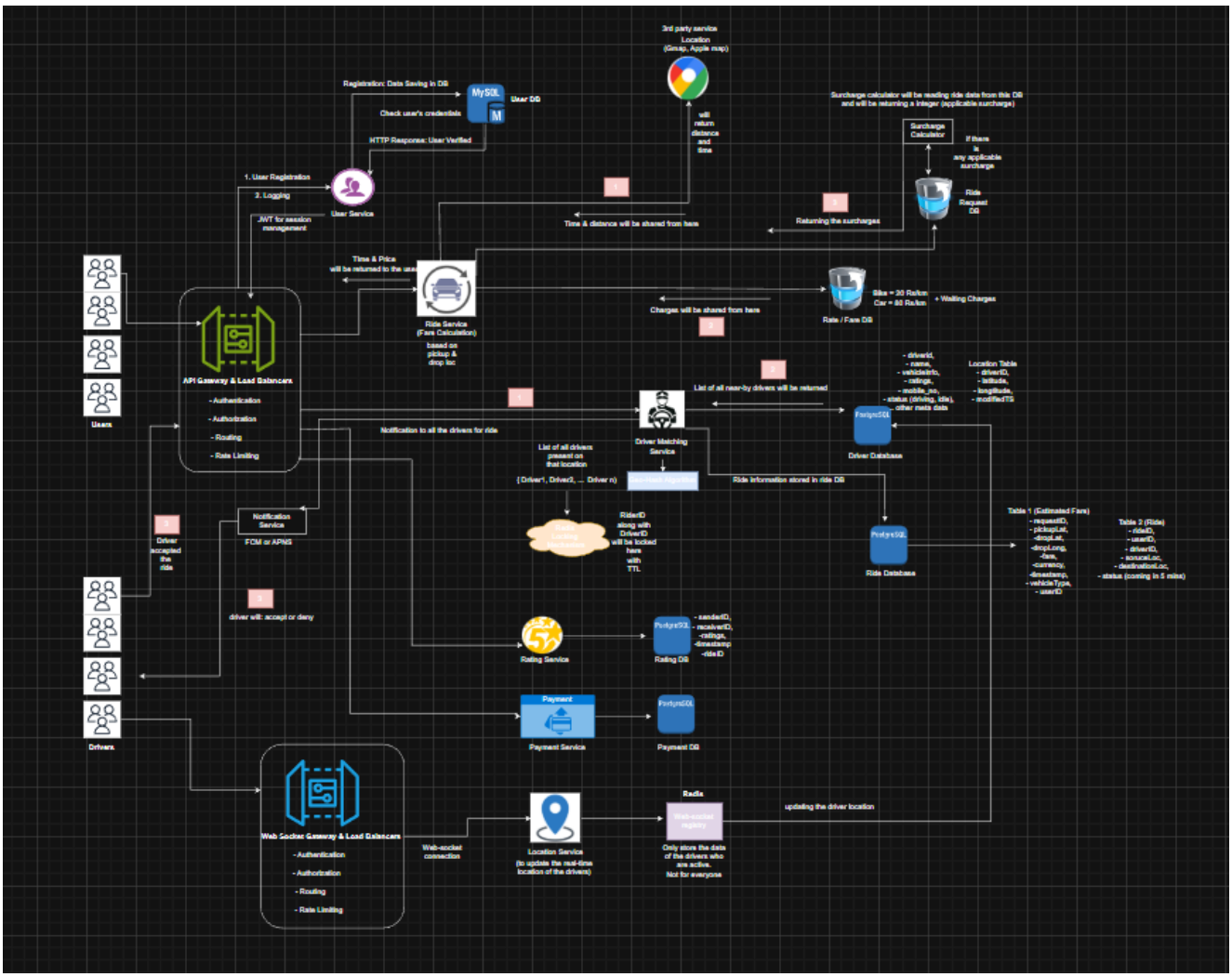
- Efficient database indexing
- Media stored separately (object storage)

High Level Design (HLD)

The system consists of Client Applications, API Gateway, User Service, Subscription Service, Search Service, Video Streaming Service, Recommendation Engine, CDN, Cache, Message Queue, and Distributed Databases. Video content is stored in object storage and delivered through CDN for low latency streaming.

Outcome

- Understand large-scale distributed system architecture
- Design real-time messaging systems
- Apply scalability and load balancing concepts
- Understand microservices architecture
- Implement messaging queues and event-driven systems
- Design secure communication systems
- Analyze trade-offs between consistency and availability
- Learn database partitioning and replication
- Understand real-time protocols (WebSockets)
- Apply system design best practices used in industry


3rd party service
Location
(Gmap, Apple map)
Registration: Data Saving in DB
MySQL
User DB
Check user's credentials
HTTP Response: User Verified
will return distance and time
Surcharge calculator will be reading ride data from this DB and will be returning a integer (applicable surcharge)
Surcharge Calculator
If there is any applicable surcharge
Ride Request DB
1. User Registration
2. Logging
User Service
JWT for session management
Time & distance will be shared from here
Returning the surcharges
Time & Price will be returned to the user
Ride Service (Fare Calculation)
based on pickup & drop loc
Charges will be shared from here
Bike = 30 Rs/km
Car = 80 Rs/km
+ Waiting Charges
Rate / Fare DB
Users
API Gateway & Load Balancers
- Authentication
- Authorization
- Routing
- Rate Limiting
List of all near-by drivers will be returned
- driverId,
- name,
- vehicleInfo,
- ratings,
- mobile_no,
- status (driving, idle),
- other meta data
Location Table
- driverID,
- latitude,
- longitude,
- modifiedTS
Notification to all the drivers for ride
Driver Matching Service
Driver Database
List of all drivers present on that location
{ Driver1, Driver2, ... Driver n}
Ride Locking Mechanism
RiderID along with DriverID will be locked here with TTL
Ride Information stored in ride DB
Notification Service
FCM or APNS
Driver accepted the ride
driver will: accept or deny
Ride Database
Table 1 (Estimated Fare)
- requestID,
- pickupLat,
-dropLat,
-dropLong,
-fare,
-currency,
-timestamp,
- vehicleType,
- userID
Table 2 (Ride)
- rideID,
- userID,
- driverID,
- sourceLoc,
- destinationLoc,
- status (coming in 5 mins)
Rating Service
Rating DB
- senderID,
- receiverID,
-ratings,
-timestamp
-rideID
Drivers
Payment
Payment Service
Payment DB
Web Socket Gateway & Load Balancers
- Authentication
- Authorization
- Routing
- Rate Limiting
Web-socket connection
Location Service
(to update the real-time location of the drivers)
Redis
Web-socket registry
Only store the data of the drivers who are active.
Not for everyone
updating the driver location